

Credibility: evaluating what's been learned

5

CHAPTER OUTLINE

5.1 Training and Testing	163
5.2 Predicting Performance	165
5.3 Cross-Validation	167
5.4 Other Estimates	169
Leave-One-Out	169
The Bootstrap	169
5.5 Hyperparameter Selection	171
5.6 Comparing Data Mining Schemes	172
5.7 Predicting Probabilities	176
Quadratic Loss Function	177
Informational Loss Function	178
Remarks	179
5.8 Counting the Cost	179
Cost-Sensitive Classification	182
Cost-Sensitive Learning	183
Lift Charts	183
ROC Curves	186
Recall-Precision Curves	190
Remarks	190
Cost Curves	192
5.9 Evaluating Numeric Prediction	194
5.10 The MDL Principle	197
5.11 Applying the MDL Principle to Clustering	200
5.12 Using a Validation Set for Model Selection	201
5.13 Further Reading and Bibliographic Notes	202

Evaluation is the key to making real progress in data mining. There are lots of ways of inferring structure from data: we have encountered many already and will see further refinements, and new methods, throughout the rest of this book. But to determine which ones to use on a particular problem we need systematic

ways to evaluate how well different methods work and to compare one with another. But evaluation is not as simple as it might appear at first sight.

What's the problem? We have the training set; surely we can just look at how well different methods do on that. Well, no: as we will see very shortly, performance on the training set is definitely not a good indicator of performance on an independent test set. We need ways of predicting performance bounds in practice, based on experiments with whatever data can be obtained.

When a vast supply of data is available, this is no problem: just make a model based on a large training set, and try it out on another large test set. But although data mining sometimes involves “big data”—particularly in marketing, sales, and customer support applications—it is often the case that labeled data, quality data, is scarce. The oil slicks mentioned in [Chapter 1](#), What's it all about?, had to be detected and marked manually—a skilled and labor-intensive process—before being used as training data. Even in the credit card application, there turned out to be only 1000 training examples of the appropriate type. The electricity supply data went back 15 years, 5000 days—but only 15 Christmas days and Thanksgivings, and just 4 February 29s and presidential elections. The electromechanical diagnosis application was able to capitalize on 20 years of recorded experience, but this yielded only 300 usable examples of faults. Marketing and sales applications certainly involve big data, but many others do not: training data frequently relies on specialist human expertise—and that is always in short supply.

The question of predicting performance based on limited data is an interesting one. We will encounter many different techniques, of which one—repeated cross-validation—is probably the method of choice in most practical limited-data situations. Comparing the performance of different machine learning schemes on a given problem is another matter that is not so easy as it sounds: to be sure that apparent differences are not caused by chance effects, statistical tests are needed.

So far we have tacitly assumed that what is being predicted is the ability to classify test instances accurately; however, some situations involve predicting class probabilities rather than the classes themselves, and others involve predicting numeric rather than nominal values. Different methods are needed in each case. Then we look at the question of cost. In most practical machine learning situations the cost of a misclassification error depends on the type of error it is—whether, e.g., a positive example was erroneously classified as negative or vice versa. When doing machine learning, and evaluating its performance, it is often essential to take these costs into account. Fortunately, there are simple techniques to make most learning schemes cost sensitive without grappling with the internals of the algorithm. Finally, the whole notion of evaluation has fascinating philosophical connections. For 2000 years philosophers have debated the question of how to evaluate scientific theories, and the issues are brought into sharp focus by machine learning because what is extracted is essentially a “theory” of the data.

5.1 TRAINING AND TESTING

For classification problems, it is natural to measure a classifier's performance in terms of the *error rate*. The classifier predicts the class of each instance: if it is correct, that is counted as a *success*; if not, it is an *error*. The error rate is just the proportion of errors made over a whole set of instances, and it measures the overall performance of the classifier.

Of course, what we are interested in is the likely future performance on new data, not the past performance on old data. We already know the classifications of each instance in the training set, which after all is why we can use it for training. We are not generally interested in learning about those classifications—although we might be if our purpose is data cleansing rather than prediction. So the question is, is the error rate on old data likely to be a good indicator of the error rate on new data? The answer is a resounding *no*—not if the old data was used during the learning process to train the classifier.

This is a surprising fact, and a very important one. Error rate on the training set is *not* likely to be a good indicator of future performance. Why? Because the classifier has been learned from the very same training data, any estimate of performance based on that data will be optimistic, and may be hopelessly optimistic.

We have already seen an example of this in the labor relations dataset. Fig. 1.3B was generated directly from the training data, and Fig. 1.3A was obtained from it by a process of pruning. The former is potentially more accurate on the data that was used to train the classifier, but may perform less well on independent test data because it may be overfitted to the training data. The first tree will look good according to the error rate on the training data, better than the second tree. But this does not necessarily reflect how they will perform on independent test data.

The error rate on the training data is called the *resubstitution error*, because it is calculated by resubstituting the training instances into a classifier that was constructed from them. Although it is not a reliable predictor of the true error rate on new data, it is nevertheless often useful to know.

To predict the performance of a classifier on new data, we need to assess its error rate on a dataset that played no part in the formation of the classifier. This independent dataset is called the *test set*. We assume that both the training data and the test data are representative samples of the underlying problem.

In some cases the test data might be distinct in nature from the training data. Consider, e.g., the credit risk problem from [Section 1.3](#). Suppose the bank had training data from branches in New York City and Florida and wanted to know how well a classifier trained on one of these datasets would perform in a new branch in Nebraska. It should probably use the Florida data as test data for evaluating the New York-trained classifier and the New York data to evaluate the Florida-trained classifier. If the datasets were amalgamated before training, performance on the test data would probably not be good indicator of performance on future data in a completely different state.

It is important that the test data is not used *in any way* to create the classifier. For example, some learning schemes involve two stages, one to come up with a basic structure and the second to optimize parameters involved in that structure, and separate sets of data may be needed in the two stages. Or you might try out several learning schemes on the training data and then evaluate them—on a fresh dataset, of course—to see which one works best. But none of this data may be used to determine an estimate of the future error rate. In such situations people often talk about three datasets: the *training* data, the *validation* data, and the *test* data. The training data is used by one or more learning schemes to come up with classifiers. The validation data is used to optimize parameters of those classifiers, or to select a particular one. Then the test data is used to calculate the error rate of the final, optimized, method. Each of the three sets must be chosen independently: the validation set must be different from the training set to obtain good performance in the optimization or selection stage, and the test set must be different from both to obtain a reliable estimate of the true error rate.

It may be that once the error rate has been determined, the test data is bundled back into the training data to produce a new classifier for actual use. There is nothing wrong with this: it is just a way of maximizing the amount of data used to generate the classifier that will actually be employed in practice. With well-behaved learning schemes, this should not decrease predictive performance. Also, once the validation data has been used—maybe to determine the best type of learning scheme to use—then it can be bundled back into the training data to retrain that learning scheme, maximizing the use of data.

If lots of data is available, there is no problem: we take a large sample and use it for training; then another, independent large sample of different data and use it for testing. Provided both samples are representative, the error rate on the test set will give a good indication of future performance. Generally, the larger the training sample the better the classifier, although the returns begin to diminish once a certain volume of training data is exceeded. And the larger the test sample, the more accurate the error estimate. The accuracy of the error estimate can be quantified statistically, as we will see in [Section 5.2](#).

The real problem occurs when there is not a vast supply of data available. In many situations the training data must be classified manually—and so must the test data, of course, to obtain error estimates. This limits the amount of data that can be used for training, validation, and testing, and the problem becomes how to make the most of a limited dataset. From this dataset, a certain amount is held over for testing—this is called the *holdout* procedure—and the remainder used for training (and, if necessary, part of that is set aside for validation). There's a dilemma here: to find a good classifier, we want to use as much of the data as possible for training; to obtain a good error estimate, we want to use as much of it as possible for testing. [Sections 5.3 and 5.4](#) review widely used methods for dealing with this dilemma.

5.2 PREDICTING PERFORMANCE

Suppose we measure the error of a classifier on a test set and obtain a certain numerical error rate—say 25%. Actually, in this section we talk about success rate rather than error rate, so this corresponds to a success rate of 75%. Now, this is only an estimate. What can you say about the *true* success rate on the target population? Sure, it's expected to be close to 75%. But how close?—within 5%? 10%? It must depend on the size of the test set. Naturally we would be more confident of the 75% figure if it was based on a test set of 10,000 instances than a test set of 100 instances. But how much more confident would we be?

To answer these questions, we need some statistical reasoning. In statistics, a succession of independent events that either succeed or fail is called a *Bernoulli process*. The classic example is coin tossing. Each toss is an independent event. Let's say we always predict heads; but rather than “heads” or “tails,” each toss is considered a “success” or a “failure.” Let's say the coin is biased, but we don't know what the probability of heads is. Then, if we actually toss the coin 100 times and 75 of them are heads, we have a situation very like the one described above for a classifier with an observed 75% success rate on a test set. What can we say about the true success probability? In other words, imagine that there is a Bernoulli process—a biased coin—whose true (but unknown) success rate is p . Suppose that out of N trials, S are successes: thus the observed success rate is $f = S/N$. The question is, what does this tell you about the true success rate p ?

The answer to this question is usually expressed as a confidence interval; i.e., p lies within a certain specified interval with a certain specified confidence. For example, if $S = 750$ successes are observed out of $N = 1000$ trials, this indicates that the true success rate must be around 75%. But how close to 75%? It turns out that with 80% confidence, the true success rate p lies between 73.2% and 76.7%. If $S = 75$ successes are observed out of $N = 100$ trials, this also indicates that the true success rate must be around 75%. But the experiment is smaller, and so the 80% confidence interval for p is wider, stretching from 69.1% to 80.1%.

These figures are easy to relate to qualitatively, but how are they derived quantitatively? We reason as follows: the mean and variance of a single Bernoulli trial with success rate p are p and $p(1 - p)$, respectively. If N trials are taken from a Bernoulli process, the expected success rate $f = S/N$ is a random variable with the same mean p ; the variance is reduced by a factor of N to $p(1 - p)/N$. For large N , the distribution of this random variable approaches the normal distribution. These are all facts of statistics: we will not go into how they are derived.

The probability that a random variable X , with zero mean, lies within a certain confidence range of width $2z$ is

$$P(-z \leq X \leq z) = c.$$

For a normal distribution, values of c and corresponding values of z are given in tables printed at the back of most statistical texts. However, the tabulations

conventionally take a slightly different form: they give the confidence that X will lie outside the range, and they give it for the upper part of the range only:

$$P(X \geq z).$$

This is called a *one-tailed* probability because it refers only to the upper “tail” of the distribution. Normal distributions are symmetric, so the probabilities for the lower tail.

$$P(X \leq -z)$$

are just the same.

Table 5.1 gives an example. Like other tables for the normal distribution, this assumes that the random variable X has a mean of 0 and a variance of 1. Alternatively, you might say that the z figures are measured in *standard deviations from the mean*. Thus the figure for $P(X \geq z) = 5\%$ implies that there is a 5% chance that X lies more than 1.65 standard deviations above the mean. Because the distribution is symmetric, the chance that X lies more than 1.65 standard deviations from the mean (above or below) is 10%, or

$$P(-1.65 \leq X \leq 1.65) = 90\%.$$

All we need do now is reduce the random variable f to have zero mean and unit variance. We do this by subtracting the mean p and dividing by the standard deviation $\sqrt{p(1-p)/N}$. This leads to

$$P\left(-z < \frac{f - p}{\sqrt{p(1-p)/N}} < z\right) = c.$$

Now here is the procedure for finding confidence limits. Given a particular confidence figure c , consult Table 5.1 for the corresponding z value. To use the table you will first have to subtract c from 1 and then halve the result, so that for $c = 90\%$ you use the table entry for 5%. Linear interpolation can be used for intermediate confidence levels. Then write the inequality in the preceding expression as an equality and invert it to find an expression for p .

Table 5.1 Confidence Limits for the Normal Distribution

$P(X \geq z)$ (%)	z
0.1	3.09
0.5	2.58
1	2.33
5	1.65
10	1.28
20	0.84
40	0.25

The final step involves solving a quadratic equation. Although not hard to do, it leads to an unpleasantly formidable expression for the confidence limits:

$$p = \left(f + \frac{z^2}{2N} \pm z \sqrt{\frac{f}{N} - \frac{f^2}{N} + \frac{z^2}{4N^2}} \right) / \left(1 + \frac{z^2}{N} \right).$$

The $\pm$ in this expression gives two values for p that represent the upper and lower confidence boundaries. Although the formula looks complicated, it is not hard to work out in particular cases.

This result can be used to obtain the values in the numeric example given earlier. Setting $f = 75\%$, $N = 1000$, and $c = 80\%$ (so that $z = 1.28$) leads to the interval $[0.732, 0.767]$ for p , and $N = 100$ leads to $[0.691, 0.801]$ for the same level of confidence. Note that the normal distribution assumption is only valid for large N (say, $N > 100$). Thus $f = 75\%$ and $N = 10$ leads to confidence limits $[0.549, 0.881]$ —but these should be taken with a grain of salt.

5.3 CROSS-VALIDATION

Now consider what to do when the amount of data for training and testing is limited. The holdout method reserves a certain amount for testing, and uses the remainder for training (and sets part of that aside for validation, if required). In practical terms, it is common to hold one-third of the data out for testing and use the remaining two-thirds for training.

Of course, you may be unlucky: the sample used for training (or testing) might not be representative. In general, you cannot tell whether a sample is representative or not. But there is one simple check that might be worthwhile: each class in the full dataset should be represented in about the right proportion in the training and testing sets. If, by bad luck, all examples with a certain class were omitted from the training set, you could hardly expect a classifier learned from that data to perform well on examples of that class—and the situation would be exacerbated by the fact that the class would necessarily be overrepresented in the test set, because none of its instances made it into the training set! Instead, you should ensure that the random sampling is done in a way that guarantees that each class is properly represented in both training and test sets. This procedure is called *stratification*, and we might speak of *stratified holdout*. While it is generally well worth doing, stratification provides only a primitive safeguard against uneven representation in training and test sets.

A more general way to mitigate any bias caused by the particular sample chosen for holdout is to repeat the whole process, training and testing, several times with different random samples. In each iteration a certain proportion—say two-thirds—of the data is randomly selected for training, possibly with stratification, and the remainder used for testing. The error rates on the different iterations are averaged to yield an overall error rate. This is the *repeated holdout* method of error rate estimation.

In a single holdout procedure, you might consider swapping the roles of the testing and training data—i.e., train the system on the test data and test it on the training data—and average the two results, thus reducing the effect of uneven representation in training and test sets. Unfortunately, this is only really plausible with a 50:50 split between training and test data, which is generally not ideal—it is better to use more than half the data for training even at the expense of test data. However, a simple variant forms the basis of an important statistical technique called *cross-validation*. In cross-validation, you decide on a fixed number of *folds*, or partitions of the data. Suppose we use three. Then the data is split into three approximately equal partitions: each in turn is used for testing and the remainder is used for training. That is, use two-thirds for training and one-third for testing, and repeat the procedure three times so that in the end, every instance has been used exactly once for testing. This is called *threefold cross-validation*, and if stratification is adopted as well—which it often is—it is *stratified threefold cross-validation*.

The standard way of predicting the error rate of a learning technique given a single, fixed sample of data is to use stratified 10-fold cross-validation. The data is divided randomly into 10 parts in which the class is represented in approximately the same proportions as in the full dataset. Each part is held out in turn and the learning scheme trained on the remaining nine-tenths; then its error rate is calculated on the holdout set. Thus the learning procedure is executed a total of 10 times on different training sets (each of which have a lot in common). Finally, the 10 error estimates are averaged to yield an overall error estimate.

Why 10? Extensive tests on numerous different datasets, with different learning techniques, have shown that 10 is about the right number of folds to get the best estimate of error, and there is also some theoretical evidence that backs this up. Although these arguments are by no means conclusive, 10-fold cross-validation has become the standard method in practical terms. Tests have also shown that the use of stratification improves results slightly. Thus the standard evaluation technique in situations where only limited data is available is stratified 10-fold cross-validation. Note that neither the stratification nor the division into 10 folds has to be exact: it is enough to divide the data into 10 approximately equal sets in which the various class values are represented in approximately the right proportion. Moreover, there is nothing magic about the exact number 10: 5-fold or 20-fold cross-validation is likely to be almost as good.

A single 10-fold cross-validation might not be enough to get a reliable error estimate if the data is limited. Different 10-fold cross-validation experiments with the same learning scheme and dataset often produce different results, because of the effect of random variation in choosing the folds themselves. Stratification reduces the variation, but it certainly does not eliminate it entirely. When seeking an accurate error estimate with limited data, it is standard procedure to repeat the cross-validation process 10 times—i.e., 10 times 10-fold cross-validation—and average the results. This involves invoking the learning algorithm 100 times on datasets that are all nine-tenths the size of the original. Getting a good measure of performance is a computation-intensive undertaking.

5.4 OTHER ESTIMATES

Tenfold cross-validation is the standard way of measuring the error rate of a learning scheme on limited data; for reliable results, 10 times 10-fold cross-validation. But many other methods are used instead. Two that are particularly prevalent are *leave-one-out* cross-validation and the *bootstrap*.

LEAVE-ONE-OUT

Leave-one-out cross-validation is simply n -fold cross-validation, where n is the number of instances in the dataset. Each instance in turn is left out, and the learning scheme is trained on all the remaining instances. It is judged by its correctness on the remaining instance—1 or 0 for success or failure. The results of all n judgments, one for each member of the dataset, are averaged, and that average represents the final error estimate.

This procedure is an attractive one for two reasons. First, the greatest possible amount of data is used for training in each case, which presumably increases the chance that the classifier is an accurate one. Second, the procedure is deterministic: no random sampling is involved. There is no point in repeating it 10 times, or repeating it at all: the same result will be obtained each time. Set against this is the high computational cost, because the entire learning procedure must be executed n times and this is usually quite infeasible for large datasets. Nevertheless, leave-one-out seems to offer a chance of squeezing the maximum out of a small dataset and getting as accurate an estimate as possible.

But there is a disadvantage to leave-one-out cross-validation, apart from the computational expense. By its very nature, it cannot be stratified—worse than that, it *guarantees* a nonstratified sample. Stratification involves getting the correct proportion of examples in each class into the test set, and this is impossible when the test set contains only a single example. A dramatic, although highly artificial, illustration of the problems this might cause is to imagine a completely random dataset that contains exactly the same number of instances of each of two classes. The best that an inducer can do with random data is to predict the majority class, giving a true error rate of 50%. But in each fold of leave-one-out, the opposite class to the test instance is in the majority—and therefore the predictions will always be incorrect, leading to an estimated error rate of 100%!

THE BOOTSTRAP

The second estimation method we describe, the bootstrap, is based on the statistical procedure of sampling *with replacement*. Previously, whenever a sample was taken from the dataset to form a training or test set, it was drawn without replacement. That is, the same instance, once selected, could not be selected again. It is like picking teams for football: you cannot choose the same person twice. But

dataset instances are not like people. Most learning schemes can use the same instance twice, and it makes a difference in the result of learning if it is present in the training set twice. (Mathematical sticklers will notice that we should not really be talking about “sets” at all if the same object can appear more than once.)

The idea of the bootstrap is to sample the dataset with replacement to form a training set. We will describe a particular variant, mysteriously (but for a reason that will soon become apparent) called the *0.632 bootstrap*. For this, a dataset of n instances is sampled n times, with replacement, to give another dataset of n instances. Because some elements in this second dataset will (almost certainly) be repeated, there must be some instances in the original dataset that have not been picked: we will use these as test instances.

What is the chance that a particular instance will not be picked for the training set? It has a $1/n$ probability of being picked each time, and so a $1 - 1/n$ probability of *not* being picked. Multiply these probabilities together for a sufficient number of picking opportunities, n , and the result is a figure of

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} = 0.368$$

(where e is the base of natural logarithms, 2.7183, not the error rate!). This gives the chance of a particular instance not being picked at all. Thus for a reasonably large dataset, the test set will contain about 36.8% of the instances and the training set will contain about 63.2% of them (now you can see why it's called the *0.632 bootstrap*). Some instances will be repeated in the training set, bringing it up to a total size of n , the same as in the original dataset.

The figure obtained by training a learning system on the training set and calculating its error over the test set will be a pessimistic estimate of the true error rate, because the training set, although its size is n , nevertheless contains only 63% of the instances, which is not a great deal compared, e.g., with the 90% used in 10-fold cross-validation. To compensate for this, we combine the test-set error rate with the resubstitution error on the instances in the training set. The resubstitution figure, as we warned earlier, gives a very optimistic estimate of the true error and should certainly not be used as an error figure on its own. But the bootstrap procedure combines it with the test error rate to give a final estimate e as follows:

$$e = 0.632 \cdot e_{\text{test instances}} + 0.368 \cdot e_{\text{training instances}}$$

Then, the whole bootstrap procedure is repeated several times, with different replacement samples for the training set, and the results averaged.

The bootstrap procedure may be the best way of estimating error for very small datasets. However, like leave-one-out cross-validation, it has disadvantages that can be illustrated by considering a special, artificial situation. In fact, the very dataset we considered above will do: a completely random dataset with two classes of equal size. The true error rate is 50% for any prediction rule. But a scheme that memorized the training set would give a perfect resubstitution score

of 100%, so that $e_{\text{training instances}} = 0$, and the 0.632 bootstrap will mix this in with a weight of 0.368 to give an overall error rate of only 31.6% ($0.632 \times 50\% + 0.368 \times 0\%$), which is misleadingly optimistic.

5.5 HYPERPARAMETER SELECTION

Many learning algorithms have parameters that can be tuned to optimize their behavior. These are called “hyperparameters” to distinguish them from basic parameters such as the coefficients in linear regression models. An example is the parameter k that determines the number of neighbors used in a k -nearest-neighbor classifier. Normally, best performance on a test set is achieved by adjusting the value of this hyperparameter to suit the characteristics of the data. However, frustratingly, it is very important not to use performance on the test data to choose the best value for k ! This is because peeking at the test data to make choices automatically introduces optimistic bias in the performance score obtained from this same data. Performance on future new data will very likely be worse than the estimate.

What to do? The trick, as mentioned earlier in this chapter, is to split the original training set into a smaller, new training set and a validation set. (The split is normally done randomly.) Then the algorithm is run several times with different hyperparameter values on this reduced training set, and each of the resulting models is evaluated on the validation set. Once the hyperparameter value that gives best performance on the validation set has been determined, a final model is built by running the algorithm with that hyperparameter value on the original, full training set. Note that the test data is not involved in any of this! Only after the final model is fixed are we allowed to use the test data to obtain an estimate of the performance of this model on new, unseen data. Basically, the test data can only be used once, to establish the final performance score.

This also applies when using methods, such as cross-validation, with multiple training and testing splits. Hyperparameter choice must be based on the training set only. When applying the above parameter selection process multiple times within a cross-validation, once for each fold, it is entirely possible that the hyperparameter values will be slightly different from fold to fold. This does not matter: hyperparameter selection is part of the overall process of learning a model, and these models will normally differ from fold to fold anyway. In other words, what is being evaluated with cross-validation is the learning process, not one particular model.

There is a drawback to this process for hyperparameter selection: if the original training data is small—and the training folds in a cross-validation often are—then splitting off a validation set will further reduce the size of the set available for training; and the validation set will be small too. This means that the choice of hyperparameter may not be reliable. This is analogous to the problem with the simple holdout estimate we encountered earlier in this chapter, and the same remedy applies: use cross-validation instead. This means that a so-called “inner”

cross-validation is applied to determine the best value of the hyperparameter for each fold of the “outer” cross-validation used to obtain the final performance estimate for the learning algorithm.

This kind of nested cross-validation process is expensive, particularly considering that the inner cross-validation must be run for each value of the hyperparameter that we want to evaluate. Things become worse with multiple hyperparameters. If a grid search is used to find the best parameter value, then with two hyperparameters and a 10×10 grid 100 inner cross-validations are needed, and this must be done for each fold of the outer cross-validation. Assuming that 10 folds are used for both the inner and outer cross-validation, the learning algorithm must be run 10 times 10 times 100 times—10,000 times! And then we might want to repeat the outer cross-validation 10 times to get a more reliable final performance estimate.

Fortunately, this process can easily be distributed across multiple computers. Nevertheless, it may be infeasible, at least with the above configuration, and it is common practice to use a smaller number of folds for the internal cross-validation, perhaps just twofolds.

5.6 COMPARING DATA MINING SCHEMES

We often need to compare two different learning schemes on the same problem to see which is the better one to use. It seems simple: estimate the error using cross-validation (or any other suitable estimation procedure), perhaps repeated several times, and choose the scheme whose estimate is smaller. And this is quite sufficient in many practical applications: if one scheme has a lower estimated error than another on a particular dataset, the best we can do is to use the former scheme's model. However, it may be that the difference is simply caused by estimation error, and in some circumstances it is important to determine whether one scheme is *really* better than another on a particular problem. This is a standard challenge for machine learning researchers. If a new learning algorithm is proposed, its proponents must show that it improves on the state of the art for the problem at hand and demonstrate that the observed improvement is not just a chance effect in the estimation process.

This is a job for a statistical test that is based on confidence bounds, the kind we met previously when trying to predict true performance from a given test-set error rate. If there were unlimited data, we could use a large amount for training and evaluate performance on a large independent test set, obtaining confidence bounds just as before. However, if the difference turns out to be significant we must ensure that this is not just because of the particular dataset we happened to base the experiment on. What we want to determine is whether one scheme is better or worse than another on average, across all possible training and test datasets that can be drawn from the domain. Because the amount of training data naturally